CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G6
System Integration and Architecture

System Design Document (SDD)

Project Title: Gym Membership & Payment System

Prepared By: Tan, Christian Aire

Version: 1

Date: February 2, 2026
Status: FINAL

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	Feb 2, 2026	Tan, Christian Aire	Initial draft	Draft
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY

1.1 Project Overview

The **Gym Membership & Payment System** is a web-based application designed to manage gym members, memberships, and payment transactions. The system allows gym members to register, log in, view their membership status, and pay membership fees through an integrated online payment system. Gym staff can manage memberships and monitor payments efficiently.

1.2 Objectives

1. To provide an easy way for gym members to manage their membership
2. To integrate an online payment system for membership fees
3. To allow gym staff to monitor payments and memberships
4. To reduce manual tracking of payments and member records

1.3 Scope

Included Features:

- User registration and login
- Membership management
- Online payment integration (e-wallet/bank)
- Dashboard showing payment status
- User profile management
- Logout functionality

Excluded Features:

- Fitness tracking
- Class scheduling
- Trainer booking
- Push notifications

1.0 INTRODUCTION

1.1 Purpose

This document describes the system design of the Gym Membership & Payment System. It includes functional and non-functional requirements, system architecture, database design, and development plan to guide system implementation.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Gym Membership & Payment System
Domain: Fitness / Membership Management
Primary Users:

1. Gym Members
2. Gym Staff

Problem Statement: Manual tracking of gym memberships and payments is time-consuming and error-prone.
Solution: A centralized system that manages memberships and integrates online payment processing.

2.2 Core User Journeys

Journey 1:Gym Member Registration & Login

1. User opens the system
2. Registers using email and password

3. Logs in to the system
4. Views dashboard and membership status
5. Places order and receives confirmation

Journey 2:Membership Payment

1. User logs in
2. Views payment status
3. Selects membership plan
4. Pays using e-wallet or bank
5. Payment status is updated

Journey 3:Staff Membership Management

1. Staff logs in
2. Views list of gym members
3. Adds, updates, or removes memberships
4. Monitors payment records

2.3 Feature List (MoSCoW)

MUST HAVE

1. User registration and login
2. Membership management
3. Payment management (CRUD)
4. Dashboard showing payment status
5. User profile
6. Logout

SHOULD HAVE

1. Payment history for members to view past transactions
2. Membership expiration tracking with visible expiry dates
3. Downloadable payment receipts
4. Basic search and filter for payment records
5. Membership status indicator (Active / Expired / Pending)

COULD HAVE

1. Email reminders for upcoming membership expiration
2. Membership renewal notifications

3. Monthly payment summary sent to users
4. Admin reports for total payments collected
5. Simple dashboard analytics

WON'T HAVE

1. Fitness progress tracking
2. Trainer booking system
3. Workout plan management
4. Diet or nutrition tracking
5. Wearable device integration

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Name, Email, Password, Confirm Password
- **Validation:** Valid email format, strong password, unique email
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
- **Security:** JWT authentication, password hashing (bcrypt)

Feature: Membership Management

- **Screens:** Membership Plans, Manage Membership
- **Display:** Membership type, duration, price, status
- **Functions:** Add, update, delete membership plans (Admin)
- **API Endpoints:** GET /memberships, POST/memberships,PUT /memberships/{id}, DELETE /memberships/{id}

Feature: Shopping Cart

- **Screens:** Cart View
- **Functions:** Add product, remove product, update quantity
- **Persistence:** Database storage per user
- **API Endpoints:** GET /cart, POST /cart/items, PUT /cart/items/{id}, DELETE /cart/items/{id}

Feature: Payment Management

- **Screens:** Payment Page, Payment History
- **Functions:** Pay membership fee, view payment records
- **Integration:** Online payment system (e-wallet/bank)

API Endpoints:POST /payments, GET /payments/history

Feature: Dashboard

Screens: User Dashboard
Display: Membership status (Active/Expired), Payment status
API Endpoints:

- GET /dashboard

Feature: Admin Panel

Screens: Member List, Payment Records
Functions: View members, monitor payments, update membership status
Access Control: Admin role required
API Endpoints:

- GET /admin/users
- GET /admin/payments

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter a valid email and password
- ➢ And confirm my password correctly
- ➢ And click “Register”
- ➢ Then my account should be created
- ➢ And I should be automatically logged in
- ➢ And redirected to the homepage

AC-2: Successful Membership Payment

- ➢ Given I am logged in
- ➢ When I select a membership plan
- ➢ And complete the payment
- ➢ Then my payment status should be updated
- ➢ And my membership should become Active

AC-3: Admin Membership Management

- ➢ Given I am logged in as an admin
- ➢ When I add or update a membership plan
- ➢ Then the changes should be visible to users

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time ≤ 2 seconds
- Page load time ≤ 3 seconds
- Supports at least 100 users at the same time
- Database queries complete within 500ms

3.2 Security Requirements

- HTTPS for all communication
- JWT authentication
- Password hashing using bcrypt
- Protection against SQL injection and XSS
- Role-based access for admin pages

3.3 Compatibility Requirements

- Works on Chrome, Firefox, Edge, Safari
- Responsive on mobile, tablet, desktop
- Compatible with Windows and macOS

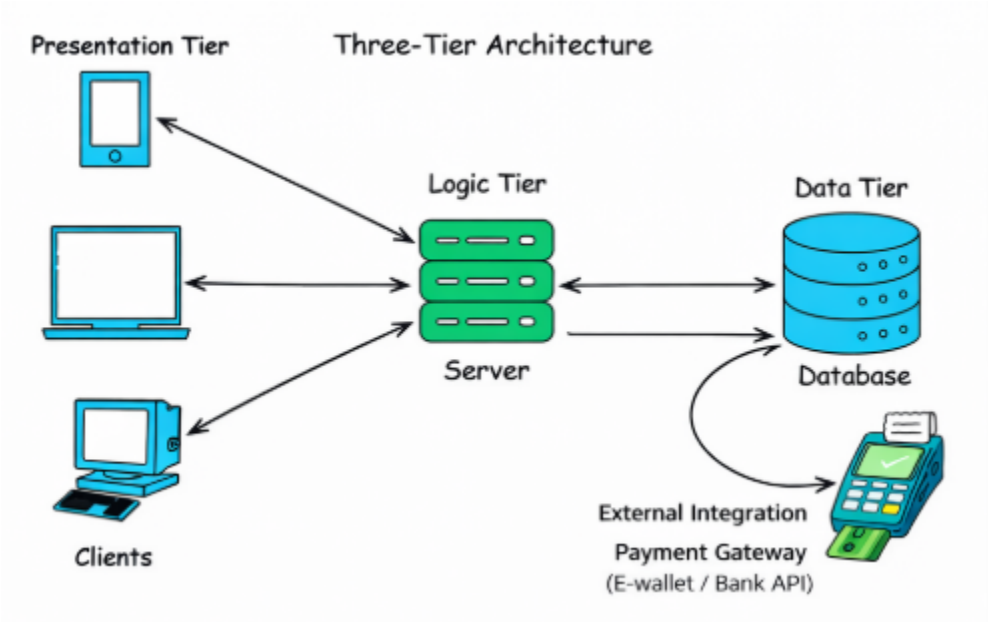
3.4 Usability Requirements

- Easy navigation
- Clear error messages
- Simple and clean interface
- Users can complete payment within 5 minutes

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** supabase
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Render

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": true,
	"data": {},
	"error": null,
	"timestamp": "2026-02-28T10:30:00Z"
	}

	{
	"success": false,
	"data": null,
	"error": {
	"code": "ERROR_CODE",
	"message": "Error message description",
	"details": "Additional error details"
	},
	"timestamp": "2026-02-28T10:30:00Z"
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": "user@email.com",
	"password": "StrongPassword123",
	"confirmPassword": "StrongPassword123",
	"firstname": "Juan",
	"lastname": "Dela Cruz"
	}

Response Structure	{
	{
	"success": true,
	"data": {
	"userId": 1,
	"email": "user@email.com",
	"role": "USER"
	},
	"error": null,
	"timestamp": "2026-02-28T10:30:00Z"
	}

User Logout

Description	User Logout
API URL	/auth/logout
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": "user@email.com",
	"password": "StrongPassword123"
	}

Response Structure	{
	"success": true,
	"data": "User successfully logged out",
	"error": null,
	"timestamp": "2026-02-28T10:30:00Z"
	}

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": "user@email.com",
	"password": "StrongPassword123"
	}

Response Structure	{
	"success": true,
	"data": {
	"accessToken": "jwt_access_token",
	"refreshToken": "jwt_refresh_token",
	"role": "USER"
	},
	"error": null,
	"timestamp": "2026-02-28T10:30:00Z"
	}

Membership Endpoints

Get Available Membership Plans

Description	Allows a user to select a membership plan.

API URL	/user/membership/select
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Required

Request Payload	{
	"membershipId": 2
	}

Response Structure	{
	"success": true,
	"data": {
	"membershipName": "Premium Plan",
	"status": "Active",
	"startDate": "2026-02-28",
	"endDate": "2027-02-28"
	},
	"error": null,
	"timestamp": "2026-02-28T10:30:00Z"
	}

Payment Endpoints

Create Payment

Description	Processes a payment for a membership
API URL	/payments
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Required

Request Payload	{
	"membershipId": 2,
	"amount": 9000.00,
	"paymentMethod": "Credit Card"
	}

Response Structure	{
	"success": true,
	"data": {
	"paymentId": 101,
	"paymentStatus": "COMPLETED",
	"paymentReference":
	"PAY-2026-0001"
	},
	"error": null,
	"timestamp": "2026-02-28T10:30:00Z"
	}
	
	"timestamp": "2026-02-28T10:30:00Z"

	}

5.3 Error Handling

HTTP Status Codes

- 200 OK – Successful request
- 201 Created – Resource created
- 400 Bad Request – Invalid input
- 401 Unauthorized – Authentication required
- 403 Forbidden – Insufficient permissions
- 404 Not Found – Resource does not exist
- 409 Conflict – Duplicate resource
- 500 Internal Server Error – Server error

Error Code Examples

Example 1	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2026-02-28T10:30:00Z"
	}

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2026-02-28T10:30:00Z"
	}

Common Error Codes

- **AUTH-001**: Invalid credentials (incorrect email/password)
- **AUTH-002**: Token expired (user's session token has expired)
- **AUTH-003**: Insufficient permissions (user tries an action they are not authorized for)
- **VALID-001**: Validation failed (missing required fields or invalid input format)
  DB-001: Resource not found (user, membership, or payment record not found)
- **DB-002**: Duplicate entry (attempted creation of a resource that already exists)
- **SYSTEM-001**: Internal server error (catch-all for server-related issues)

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)

Detailed Relationships:

- **One-to-Many**: One **User** can have multiple **Payments** (A user can make several payments)
- **One-to-Many**: One **Membership** can be linked to many **User_Memberships** (A membership plan can be assigned to multiple users)

- **One-to-One**: One **User** can have one **Active Membership** (Each user has one active membership at a time)
- **One-to-Many**: One **User** can have many **Refresh Tokens** (Used for JWT token refresh)

Key Tables:

- **users**: User accounts and authentication details
- **memberships**: Gym membership plans (e.g., Basic, Premium)
- **user_memberships**: Links users to their active membership plans
- **payments**: Records of payments made by users for memberships
- **refresh_tokens**: Stores JWT refresh tokens

Table Structure Summary:

users

- **id** (PK)
- **email** (unique)
  password_hash
- **firstname**
- **lastname**
- **role** (ENUM: USER, ADMIN)
- **created_at** (timestamp)

memberships

- **id** (PK)
- **name** (ENUM: Basic, Premium, Annual)
- **duration_months** (int)
- **price** (decimal)
- **description** (text)
- **created_at** (timestamp)

user_memberships

- **id** (PK)
- **user_id** (FK → users.id)
- **membership_id** (FK → memberships.id)
- **start_date** (timestamp)
- **end_date** (timestamp)

- **status** (ENUM: Active, Expired, Cancelled)

payments

- **id** (PK)
- **user_id** (FK → users.id)
- **membership_id** (FK → memberships.id)
- **amount** (decimal)
- **payment_reference** (varchar)
- **payment_method** (ENUM: GCASH, Credit Card, Bank Transfer)
- **payment_status** (ENUM: Pending, Completed, Failed)
- **payment_date** (timestamp)
- **refresh_tokens**
- **id** (PK)
- **user_id** (FK → users.id)
- **token** (varchar)
- **expiry_date** (timestamp)

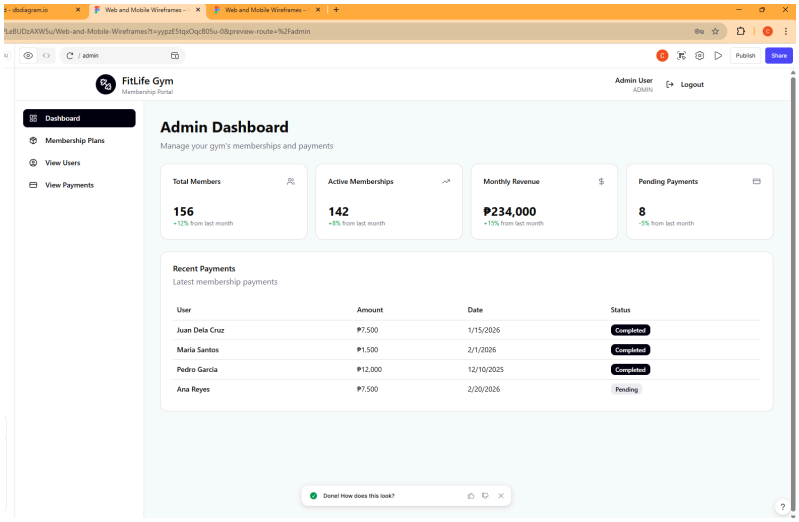
6.2 Relationships

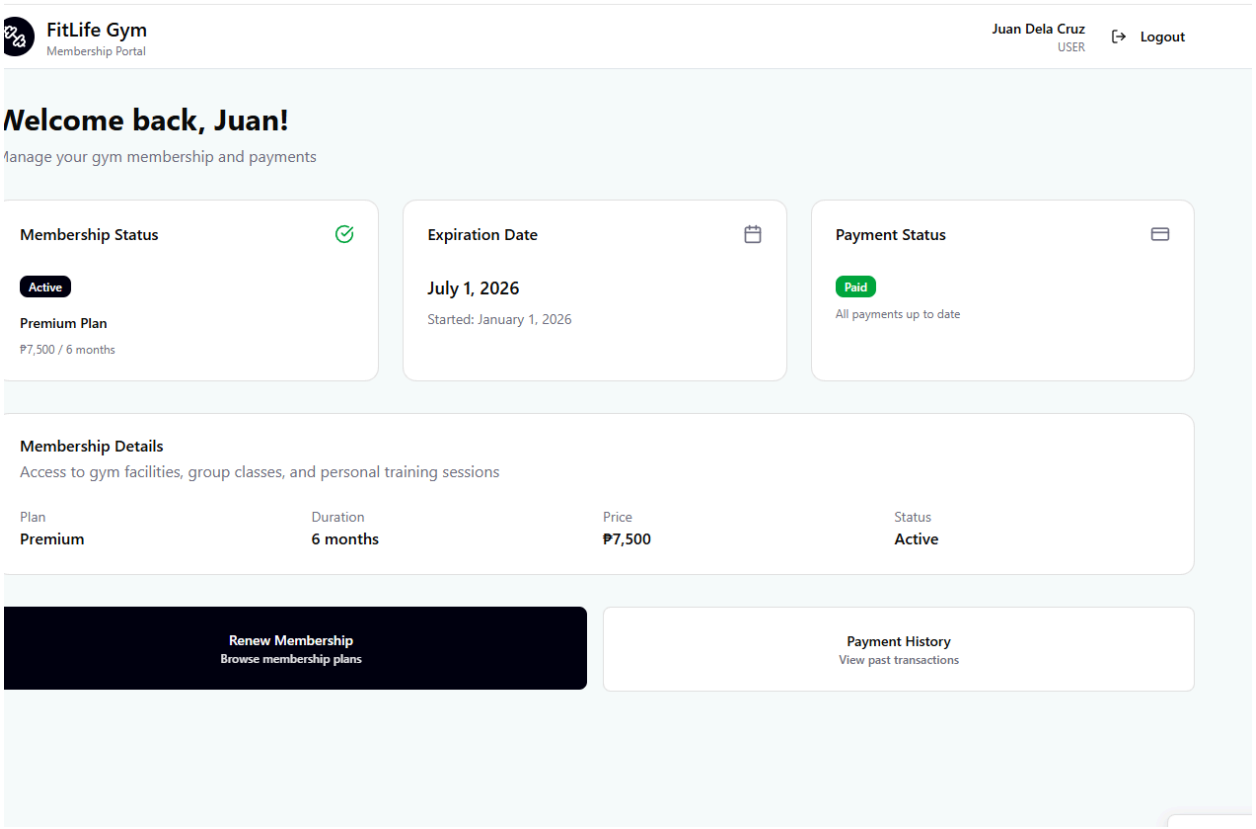
- **One-to-Many**: One **User** → Many **Payments** (A user can make several payments)
- **One-to-Many**: One **Membership** → Many **User_Memberships** (A membership plan can be assigned to many users)
- **One-to-One**: One **User** → One **Active Membership** (A user has one active membership at a time)
- **One-to-Many**: One **User** → Many **Refresh Tokens** (JWT tokens for the user)

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma





Login Page

Header: System Logo
Content:

- Email Field
- Password Field

- Login Button
- Link to Register

Registration Page

- First Name
- Last Name
- Email
- Password
- Confirm Password
- Register Button

User Dashboard

Displays:

- Membership Status (Active / Expired)
  Expiration Date
  Payment Status
- Renew Membership Button

Membership Selection Page

Displays:

- Membership Plan Name
- Price
- Duration
- Select Button

Payment Page

- Selected Membership Summary
- Payment Method Selection
- Confirm Payment Button

Payment History Page

- List of Transactions
- Date
- Amount
- Status
- Reference Number

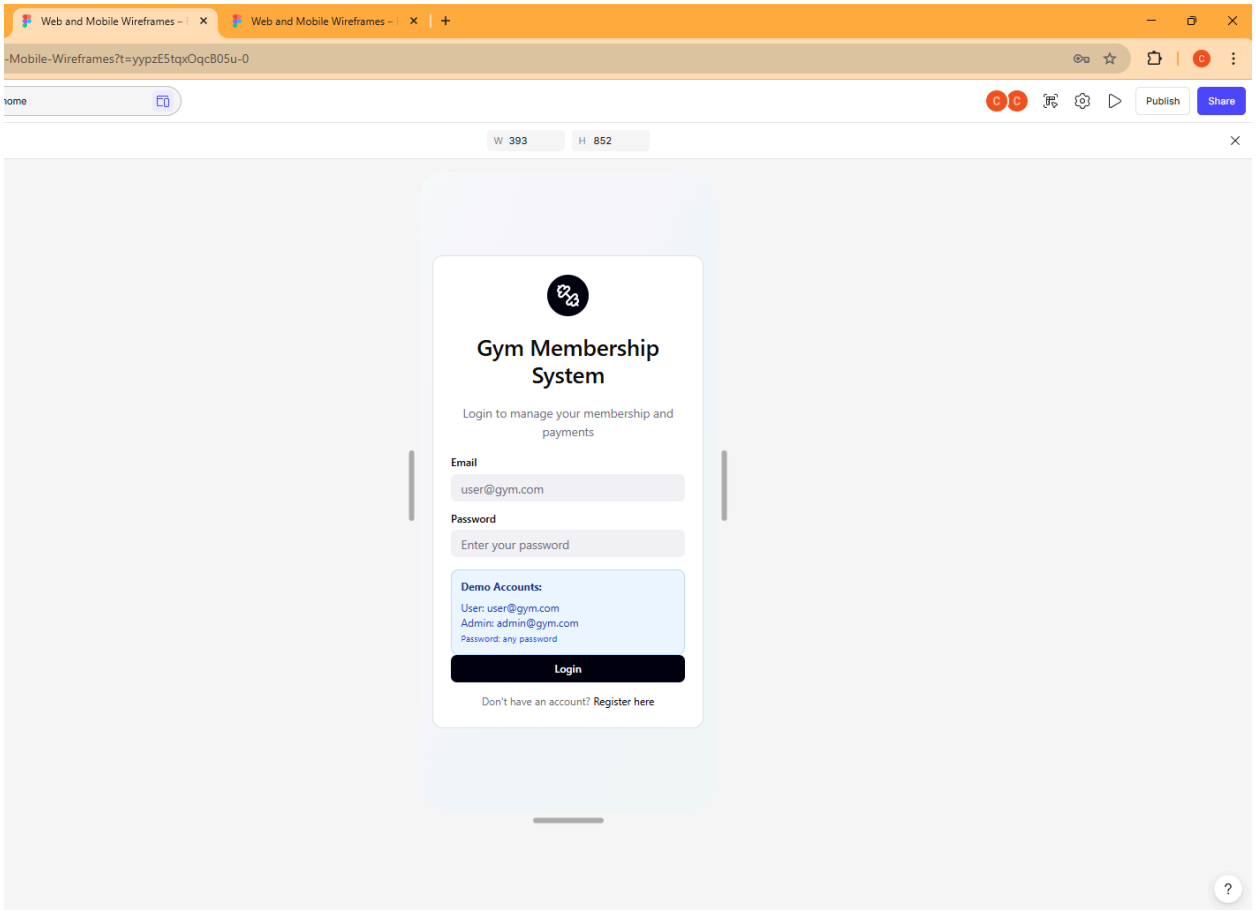
Admin Dashboard

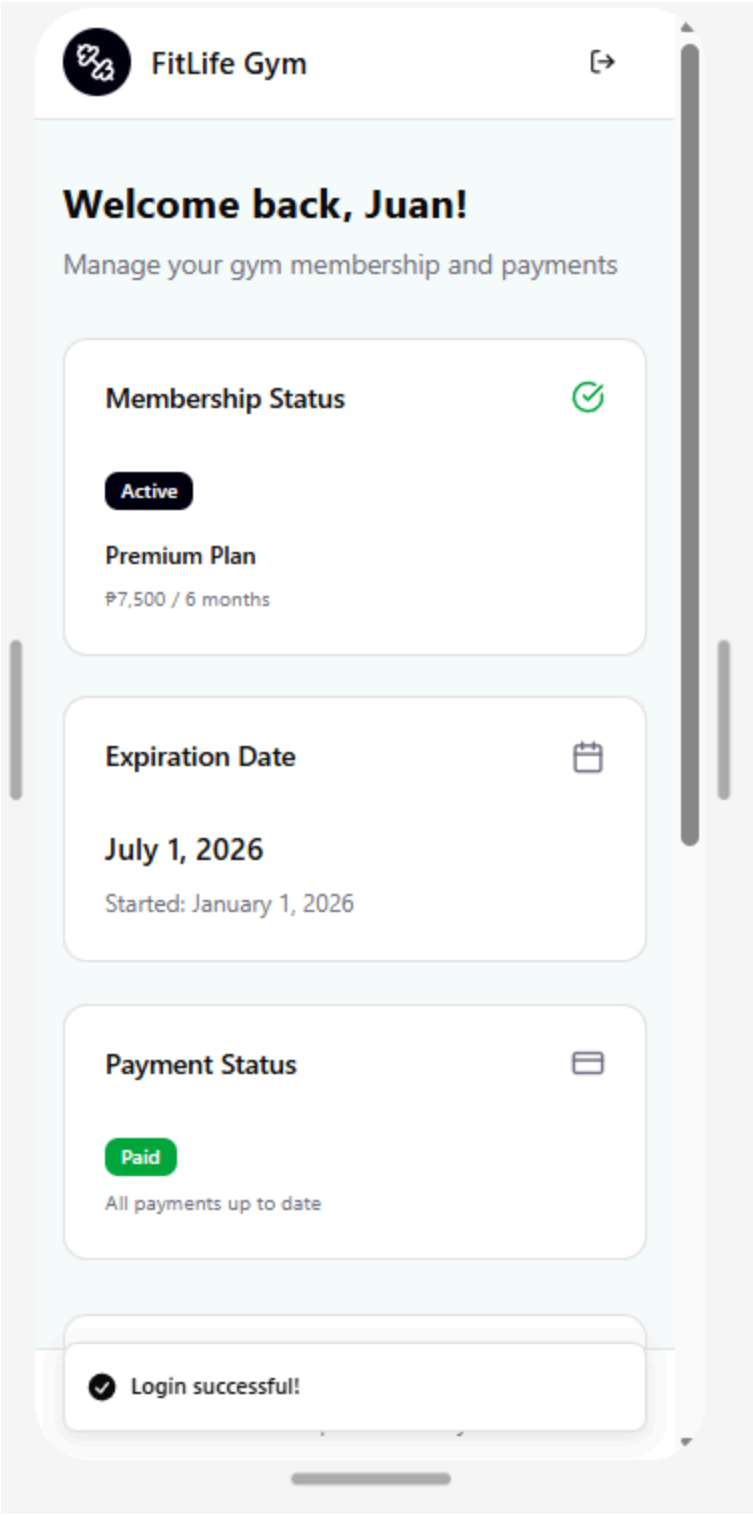
Sidebar Navigation:

- Dashboard
- Manage Membership Plans
- View Users
- View Payments

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma


Web and Mobile Wireframes –
Web and Mobile Wireframes –
-Mobile-Wireframes?t=yypzE5tqxOqcB05u-0
Publish
Share
W 393
H 852
Gym Membership System
Login to manage your membership and payments
Email
user@gym.com
Password
Enter your password
Demo Accounts:
User: user@gym.com
Admin: admin@gym.com
Password: any password
Login
Don't have an account? Register here



Bottom Navigation

Mobile Screens:

- Login Screen
- Registration Screen
- Dashboard Screen
- Membership Screen
- Payment Screen
- Admin Screen (if role = Admin)

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Responsive layouts
- Simplified forms

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

- **Day 1-2**: Project setup and documentation
- **Day 3-4**: Complete Functional Requirements Specification (FRS) and Non-Functional Requirements (NFR)
- **Day 5-7**: System architecture design (user authentication, membership plans, payment flow)

Week 2: Detailed Design

- **Day 1-2**: API specification (endpoints for registration, membership selection, payment processing)
- **Day 3-4**: Database design (users, memberships, payments, refresh tokens)
  Day 5-6: UI/UX wireframes (login, registration, membership selection, payment history)
- **Day 7**: Implementation plan finalization (breakdown of tasks and assignment)

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

- **Day 1**: Spring Boot setup with dependencies (JPA, Spring Security, JWT)
- **Day 2**: Database configuration and entity models (users, memberships, payments)
- **Day 3**: JWT authentication implementation (secure login and token-based auth)
- **Day 4**: User management endpoints (register, login, logout, refresh tokens)
  Day 5: Membership CRUD operations (adding and retrieving membership plans)

Week 4: Core Features

- **Day 1**: Membership selection functionality (selecting a membership plan)
- **Day 2**: Payment processing (handling payments via API, linking to membership)
- **Day 3**: Payment history and status tracking (viewing past payments)
- **Day 4**: Error handling and validation (proper error responses and validation)
- **Day 5**: API documentation and testing (ensure all backend endpoints are well-documented)

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

- **Day 1**: React setup with TypeScript (basic React structure, TypeScript support)

- **Day 2**: Authentication pages (login, register)
- **Day 3**: Membership listing and selection page
- **Day 4**: Membership details and plan selection page
- **Day 5**: User dashboard and payment history page

Week 6: Complete Web Features

- **Day 1**: Payment processing flow (integrating frontend with payment API)
- **Day 2**: Payment confirmation and order history
- **Day 3**: Admin dashboard (view users, memberships, and payments)
- **Day 4**: Responsive design polish (ensure web app works across devices)
- **Day 5**: API integration and testing (end-to-end integration with backend)

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

- **Day 1**: Android Studio setup and project structure
- **Day 2**: Authentication screens (login, register)
- **Day 3**: Membership browsing and selection
- **Day 4**: User dashboard and viewing payment history
- **Day 5**: API service layer (Retrofit for connecting to backend)

Week 8: Complete Mobile App

- **Day 1**: Payment flow integration (process payments from mobile app)
- **Day 2**: Order management (track and view membership status)
- **Day 3**: UI polish and animations (enhance user experience)
- **Day 4**: Testing on emulator/device (ensure functionality across devices)
- **Day 5**: APK generation and documentation (finalizing Android app)

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

- **Day 1**: End-to-end testing across platforms (web, mobile, backend integration)
- **Day 2**: Bug fixes and optimization (ensure smooth performance)
- **Day 3**: Security review (ensure no vulnerabilities)
  Day 4: Performance testing (test app speed and efficiency)
- **Day 5**: Documentation updates (finalize user and developer docs)

Week 10: Deployment

- **Day 1**: Backend deployment (Railway or Heroku)
- **Day 2**: Web app deployment (Vercel or Netlify)
- **Day 3**: Mobile APK distribution (distribute the Android app)
- **Day 4**: Final testing (verify everything is working correctly)
- **Day 5**: Project submission (submit the complete project)

Milestones:

- **M1 (End Week 2)**: All design documents complete (FRS, NFR, database schema, wireframes)
- **M2 (End Week 4)**: Backend API fully functional (user authentication, membership CRUD, payment flow)
- **M3 (End Week 6)**: Web application complete (membership selection, payment processing, user dashboard)
- **M4 (End Week 8)**: Mobile application complete (Android app with payment and membership management)
- **M5 (End Week 10)**: Full system deployed and integrated (web app, mobile app, backend deployed and tested)

Critical Path:

- **Authentication system (Week 3)**: Foundation for user login and secure access.
- **Membership selection and payment system (Week 3-4)**: Core functionality for the system.
- **Checkout process (Week 6)**: Integrating the frontend with payment API.
- **Cross-platform testing (Week 9)**: Ensure compatibility and smooth user experience across devices.

Risk Mitigation:

- Start with the simplest working version of each feature (focus on core functionality first).
- Test integration points early and often (web <-> backend, mobile <-> backend).
- Keep backup versions of working code to avoid major issues later.
- Prioritize critical features (authentication, membership management, payments) before adding extra features like advanced UI polish.